

P0682R0: Repairing elementary string conversions

Introduction

This paper proposes a few repairs for the elementary string conversions introduced into C++17 via P0067R5 "Elementary string conversions, revision 5". In particular, the following issues were raised in e-mail discussions:

- Move the functions into a separate header; do not cram them into `<utility>` (suggestion by Zhihao Yuan: `<textproc>`)
- Depending on `std::error_code` may cause an indirect dependency on `std::string`; use `std::errc` instead (LWG 2955).
- Clarify the behavior when parsing "-" as a signed number. The outcome should be `errc::invalid_argument`.
- Rounding behavior for floating-point `to_chars` / `from_chars` (Edward Catmur via std-discussion)

This paper should be treated as a Defect Report against C++17.

Discussion

Separate header

The `<utility>` header contains a number of unrelated tools; its scope should not be expanded even further. Alternative candidates among existing headers would be:

- `<string>` — but `to_chars` / `from_chars` don't use `std::string`
- `<cstring>` — but adding C++ names to headers adapted from C is probably unwise
- `<cstdlib>` — but adding C++ names to headers adapted from C is probably unwise

So, assuming consensus to create a new header is achieved, this boils down to a naming discussion for which I suggest the "expert champion" approach previously discussed. Candidate names are:

- `<textproc>` (Zhihao Yuan) — but there is not a lot of text processed
- `<textconv>` — but might be confused with encoding conversions
- `<charconv>` — but more than one character is converted
- `<stringconv>` — but might be confused with encoding conversions
- `<fromto_chars>` — a bit long and artificial
- `<chars>`

Dependency on `std::error_code` (LWG 2955)

Raised by Billy Robert O'Neal III in <http://lists.isocpp.org/lib/2017/03/2400.php>.

This dependency brings in functions returning a `std::string`, which is at a higher level than the functions `to_chars` and `from_chars`. For a low-level facility which might also be made available on freestanding environments, such a dependency seems to violate boundaries between abstraction levels.

The recommendation is to directly use the `std::errc` enumeration (22.5.1 [system_error.syn]), However, the usage pattern of the functions deteriorates to something like:

```
if (auto [ptr, ec] = to_chars(p, last, 42); ec != std::errc()) {  
    // failure case  
}
```

Clarify the behavior when parsing "-" as a signed number

This is a near-editorial clarification.

Rounding behavior for floating-point to_chars / from_chars

This topic only concerns "shortest string representation" conversions, i.e. those without a precision. The round-trip requirements already remove some freedom in this area.

"For example `0x1.00000000000001p0` is approx. `1.00000000000000222045`, so is `1.0000000000000003` an acceptable output from `to_chars`, or only `1.0000000000000002`?" The suggested resolution is to prefer the output that has the smallest difference to the original floating-point value.

Should the current rounding mode be considered in `from_chars`? For example, `1e23` might be read as `0x1.52d02c7e14af6p76` or as `0x1.52d02c7e14af7p76`. If the current rounding mode is, in fact, considered, the follow-on question is whether the round-trip guarantee only applies when both `to_chars` and `from_chars` operate under the same rounding mode, or regardless of rounding mode. For applications such as interval arithmetic, it seems worthwhile if `from_chars` does consider the rounding mode, but `to_chars` should certainly not be forced to output a long digit string for `1e23` to disambiguate.

Wording changes

Change in 23.2.1 [utility.syn]:

```
struct to_chars_result {  
    char* ptr;  
    error_code errc ec;  
};  
[...]  
struct from_chars_result {  
    const char* ptr;  
    error_code errc ec;  
};
```

Change in 23.2.8 [utility.to.chars] paragraphs 1 and 2:

... If the member `ec` of the return value is such that the value, ~~when converted to bool, is false~~ **is equal to the value of a value-initialized `errc`**, the conversion was successful and the member `ptr` is the one-past-the-end pointer of the characters written. ..

The functions that take a floating-point value but not a precision parameter ensure that the string representation consists of the smallest number of characters such that there is at least one digit before the radix point (if present) and parsing the representation using the corresponding `from_chars` function **using the same rounding mode (29.4 [cfenv])** recovers value exactly. **If there are several such representations, the representation with the smallest difference to the floating-point argument value is chosen, resolving any remaining ties using the current rounding mode.**

[Note: ...]

Change in 23.2.9 [utility.from.chars] paragraph 1:

All functions named `from_chars` analyze the string `[first, last)` for a pattern, where `[first, last)` is required to be a valid range. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `errc::invalid_argument`. **[Note: If the pattern allows for an optional sign, but the string has no digit characters following the sign, no characters match the pattern.]** ... Otherwise, `value` is set to the parsed value, **after rounding according to the current rounding mode (29.4 [cfenv])**, and the member `ec` is ~~set such that the~~ **conversion to bool yields false** **value-initialized**.